COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	8/24/2026	**DATE SUBMITTED:**	8/24/2026
NAME:	Momo, Borge Rudolf A.	**INSTRUCTOR:**	Engr. Quejado

ACTIVITY NO. 4 **Running Elevated Ad hoc Commands**

1. PROCEDURE OUTPUT

Task 1: Run elevated ad hoc commands

1. Locally, we use the command sudo apt update when we want to download package information from all configured resources. The sources are often defined in /etc/apt/sources.list file and other files located in /etc/apt/sources.list.d/ directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

```
server1 | FAILED! => {
    "changed": false,
    "msg": "Task failed: Cannot resolve 'update_cache=true' to an action or modu
le."
}
server2 | FAILED! => {
    "changed": false,
    "msg": "Task failed: Cannot resolve 'update_cache=true' to an action or modu
le."
}
```

What is the result of the command? Is it successful?

- It was basically stating that the servers are failing to connect..

Try editing the command and add something that would elevate the privilege. Issue the command ansible all -m apt -a update_cache=true --become --ask-become-pass. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The update_cache=true is the same thing as running sudo apt update. The --become command elevates the privileges and the --ask-become-pass asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

```
momo1@workstation:~/ansible$ ansible all -m apt -a "update_cache=true" --become
server2 | CHANGED => {
    "cache_update_time": 1787566613,
    "cache_updated": true,
    "changed": true
}
server1 | CHANGED => {
    "cache_update_time": 1787566613,
    "cache_updated": true,
    "changed": true
}
momo1@workstation:~/ansible$
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: ansible all -m apt -a name=vim-nox --become --ask-become-pass. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively. Was the command successful?

```
momo1@workstation:~/ansible$ ansible all -m apt -a "name=vim-nox" --become
server1 | SUCCESS => {
    "cache_update_time": 1787567520,
    "cache_updated": false,
    "changed": false
}
server2 | SUCCESS => {
    "cache_update_time": 1787566613,
    "cache_updated": false,
    "changed": false
}
momo1@workstation:~/ansible$ sudo apt search vim-nox
[sudo: authenticate] Password:
vim-nox/resolute-updates,resolute-security 2:9.1.2141-1ubuntu4.8 amd64
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [in>
  Vi IMproved - enhanced vi editor - compact version
```

- Yes, the command was successful.

b. Check the logs in the servers using the following commands: cd /var/log. After this, issue the command ls, go to the folder apt and open history.log. Describe what you see in the history.log.

```
momo1@workstation: ~/ansi | momo1@workstation: ~/ansi | momo1@workstation: /va ×
 GNU nano 8.7.1                    history.log

Start-Date: 2026-08-24  09:19:07
Commandline: /usr/bin/unattended-upgrade
Upgrade: wget:amd64 (1.25.0-2ubuntu4, 1.25.0-2ubuntu4.4)
End-Date: 2026-08-24  09:19:10

Start-Date: 2026-08-24  09:19:13
Commandline: /usr/bin/unattended-upgrade
Upgrade: openssl-provider-legacy:amd64 (3.5.5-1ubuntu3.2, 3.5.5-1ubuntu3.3), li>
End-Date: 2026-08-24  09:19:16
```

- The contents are the previous command lines I have entered along with the dates.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.
a. Issue the command: ansible all -m apt -a name=snapd --become --ask-become-pass
Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

```
momo1@workstation:~/ansible$ ansible all -m apt -a name=snapd --become --ask-bec
ome-pass
BECOME password:
server1 | SUCCESS => {
    "cache_update_time": 1787567520,
    "cache_updated": false,
    "changed": false
}
server2 | SUCCESS => {
    "cache_update_time": 1787567735,
    "cache_updated": false,
    "changed": false
}
momo1@workstation:~/ansible$
```

- The results were the same as ansible all -m apt -a name=vim-nox --become --ask-become-pass.

b. Now, try to issue this command: ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass

```
server2 | CHANGED => {
    "cache_update_time": 1787567735,
    "cache_updated": false,
    "changed": true,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "Reading package lists...\nBuilding dependency tree...\nReading st
ate information...\nSolving dependencies...\nThe following packages will be upgr
aded:\n  snapd\n1 upgraded, 0 newly installed, 0 to remove and 115 not upgraded.
\nNeed to get 0 B/42.6 MB of archives.\nAfter this operation, 659 kB of addition
al disk space will be used.\n(Reading database ... \r(Reading database ... 5%\r(
Reading database ... 10%\r(Reading database ... 15%\r(Reading database ... 20%\r
(Reading database ... 25%\r(Reading database ... 30%\r(Reading database ... 35%\
r(Reading database ... 40%\r(Reading database ... 45%\r(Reading database ... 50%
\r(Reading database ... 55%\r(Reading database ... 60%\r(Reading database ... 65
%\r(Reading database ... 70%\r(Reading database ... 75%\r(Reading database ... 8
0%\r(Reading database ... 85%\r(Reading database ... 90%\r(Reading database ...
95%\r(Reading database ... 100%\r(Reading database ... 200750 files and director
ies currently installed.)\r\nPreparing to unpack .../snapd_2.76.3+ubuntu26.04_am
d64.deb ...\r\nUnpacking snapd (2.76.3+ubuntu26.04) over (2.76+ubuntu26.04.3) ..
.\r\nSetting up snapd (2.76.3+ubuntu26.04) ...\r\nsnapd.failure.service is a dis
abled or a static unit not running, not starting it.\r\nsnapd.gpio-chardev-setup
.target is a disabled or a static unit not running, not starting it.\r\nsnapd.sn
ap-repair.service is a disabled or a static unit not running, not starting it.\r
\nProcessing triggers for gnome-menus (3.38.1-1ubuntu1) ...\r\nProcessing trigge
rs for man-db (2.13.1-1build1) ...\r\nProcessing triggers for dbus (1.16.2-2ubun
tu4) ...\r\nProcessing triggers for desktop-file-utils (0.28-1build1) ...\r\n",
    "stdout_lines": [
        "Reading package lists...",
        "Building dependency tree...",
        "Reading state information...",
        "Solving dependencies...",
        "The following packages will be upgraded:",
        "  snapd"
```



Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

- It basically checked for the latest version of snapd on the remote servers and updated it if a newer version was available.

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command nano install_apache.yml. This will create a playbook file called install_apache.yaml. The .yaml is the basic standard extension for playbook files.
When the editor appears, type the following:
Make sure to save the file. Take note also of the alignments of the texts. Use TAB instead of SPACE BAR for spacing.

```
momo1@workstation: ~/ansible    momo1@workstation: ~/a
GNU nano 8.7.1                  install_apache
---
- hosts: all
  become: true
  tasks:

  - name: install apache2 package
    apt:
      name: apache2
```

2. Run the yml file using the command: ansible-playbook --ask-become-pass install_apache.yml. Describe the result of this command.

```
momo1@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [server1]
ok: [server2]

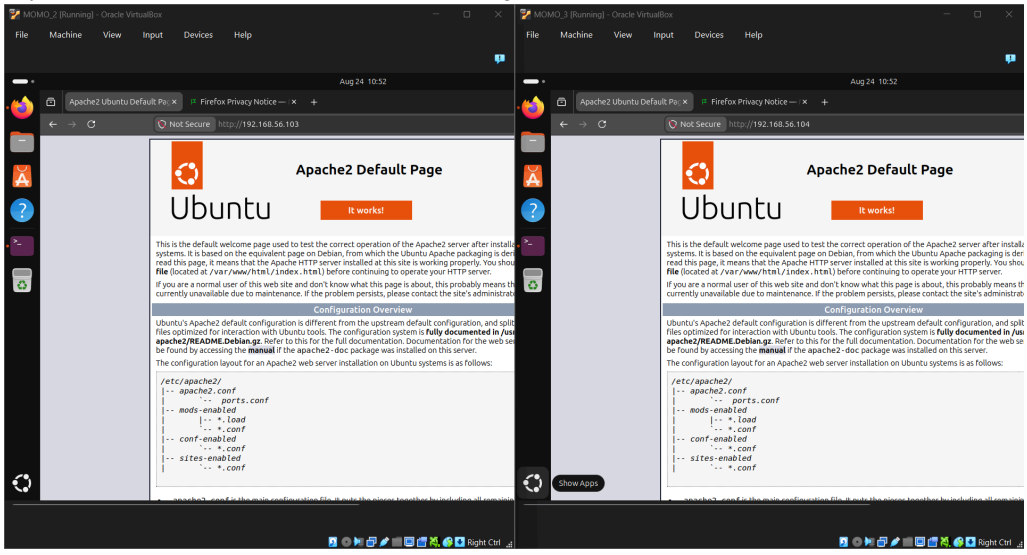
TASK [install apache2 package] *************************************************************
changed: [server2]
changed: [server1]

PLAY RECAP *********************************************************************************
server1                    : ok=2    changed=1    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0
server2                    : ok=2    changed=1    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0

momo1@workstation:~/CPE212_MOMO$
```

- It executed the task that I included in the install_apache.yml which worked properly for both servers.

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.



4. Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized. What is the output?

```
 GNU nano 8.7.1                                  ir
---
- hosts: all
  become: true
  tasks:

  - name: install totally not apache2
    apt:
      name: apache2
```

```
TASK [install totally not apache2] ****************************************************
ok: [server1]
ok: [server2]

PLAY RECAP ********************************************************************************
server1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0
server2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0
```

- It still worked even if the name of the task was different, as long as the target package name was correct.

5. This time, we are going to put additional tasks into our playbook. Edit the install_apache.yml. As you can see, we are now adding an additional command, which is the update_cache. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.
Save the changes to this file and exit.

```
momo1@workstation: ~/ansible | momo1@workstation: ~/ansible
 GNU nano 8.7.1                          install_apache.yml
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2
```

6. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
momo1@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [server1]
ok: [server2]

TASK [update repository index] *************************************************************
changed: [server1]
changed: [server2]

TASK [install apache2 package] *************************************************************
ok: [server1]
ok: [server2]

PLAY RECAP *********************************************************************************
server1                    : ok=3    changed=1    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0
server2                    : ok=3    changed=1    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0

momo1@workstation:~/CPE212_MOMO$
```

- We can see that the yellow text “changed=1” was set to 1 meaning there were changes on the remote servers.

7. Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier.
Save the changes to this file and exit.

```
momo1@workstation: ~/ansible    momo
 GNU nano 8.7.1
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
```

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
momo1@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [server1]
ok: [server2]

TASK [update repository index] *************************************************************
changed: [server1]
changed: [server2]

TASK [install apache2 package] *************************************************************
ok: [server1]
ok: [server2]

TASK [add PHP support for apache] **********************************************************
changed: [server1]
changed: [server2]

PLAY RECAP *********************************************************************************
server1                    : ok=4    changed=2    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0
server2                    : ok=4    changed=2    unreachable=0    failed=0    skipped=0    r
escued=0    ignored=0

momo1@workstation:~/CPE212_MOMO$
```

- We can see that the update repository index and add PHP support for apache tasks had some changes.

9. Finally, make sure that we are in sync with GitHub. Provide the link to your GitHub repository.
   - https://github.com/BorgeMomo/CPE212_MOMO

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?

- Using a playbook is important since it allows us to execute as well as organize the multiple tasks in Ansible. Instead of typing commands one by one on different servers, we can put the tasks in a file and run them together. It also makes the process easier to repeat and manage on multiple servers.

2. Summarize what we have done on this activity.

- For this activity, I set up an Ansible control node and connected it to two remote servers using SSH which was already done from previous activities. I configured the inventory file to identify the remote servers and tested their connectivity using Ansible. I also created and ran a playbook that automated tasks such as installing packages and creating files or directories on the remote servers.

Conclusions/Learnings

- In conclusion, I learned that Ansible can manage multiple servers from one control node and I also learned how the inventory files and playbooks work together to connect to and manage remote servers. I also learned that playbooks make server management easier because tasks can be automated instead of being done manually.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Alternative methods to the procedures.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **In my End, the commands in the procedures were not executing properly and I had to modify it by searching online some different methods and eventually made it work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.